

NAME: TANKISO MASOEBE

PRGRAMMING WITH C++WK3

1.SCENARIO ANALYSIS:

Inheritance Scenario

In a game system, inheritance can be used to create different types of characters that share common features. For example, a game can have a parent class called Character that contains common properties such as name, health, speed, and methods such as move() and attack(). From this parent class, different child classes can be created, such as Warrior, Archer, and Mage. The Warrior class can inherit the basic features of Character and add a special ability such as swordAttack(). The Archer class can inherit the same features but add shootArrow(), while the Mage can add castSpell(). This allows programmers to reuse existing code instead of writing the same properties and methods repeatedly. Inheritance also makes the game system easier to maintain because common features are controlled by the parent class, while each child class can have its own unique abilities and behavior.

2.CONCEPT RESEARCH:

Multiple Inheritance

Multiple inheritance is an object-oriented programming concept where a single class inherits properties and methods from two or more parent classes. It allows programmers to combine features from different classes into one child class. For example, in a game system, a FlyingWarrior class could inherit abilities from both a Warrior class and a FlyingCharacter class. Multiple inheritance can reduce code duplication and make programs more flexible by allowing a class to reuse features from different sources. However, it can also create complexity when parent classes contain methods with the same name. Programmers must carefully manage these conflicts to keep the program clear and reliable.

3.TOOL PRACTICE:

```
#include <iostream>
```

```
using namespace std;
```

```
// Parent class
```

```
class Animal {
```

```
public:
```

```
void eat() {  
    cout << "The animal is eating." << endl;  
}  
};
```

```
// Child class
```

```
class Dog : public Animal {  
public:  
    void bark() {  
        cout << "The dog is barking." << endl;  
    }  
};
```

```
int main() {  
    Dog myDog;  
  
    myDog.eat(); // Inherited method  
    myDog.bark(); // Dog's own method  
  
    return 0;  
}
```

Reflection

Writing the C++ inheritance program helped me understand how inheritance allows one class to use the properties and methods of another class. In the program, the Animal class is the parent class, while the Dog class is the child class. The Dog class inherits the eat() method from Animal and also has its own bark() method. I learned that inheritance helps reduce repeated code and makes programs easier to organize and maintain. Using Visual Studio Code also helped me practice writing, saving, compiling, and running a C++ program. Overall, this exercise improved my understanding of object-oriented programming and inheritance.

4.UML DIAGRAM:

Explanation

The UML diagram shows an inheritance hierarchy using an Animal parent class and two child classes, Dog and Cat. The Animal class contains common attributes such as name and age, as well as methods such as eat() and sleep(). The Dog and Cat classes inherit these common features from Animal. Each child class also has its own special attribute and method. Dog has a breed and bark() method, while Cat has a color and meow() method. This diagram demonstrates how inheritance allows classes to share common features while still having their own unique characteristics. It also helps programmers understand the relationship between classes clearly.

